

SENAI -DF
Curso Técnico em Desenvolvimento de Sistemas

Python para Web

HTTP -- FastAPI -- Pydantic -- Rotas -- Swagger

2026

Desenvolvimento de Sistemas | Módulo 2 | 2026

Capítulo 0

Apresentação do Módulo

No Módulo 1 você construiu interfaces completas com HTML, CSS e JavaScript, e aprendeu a consumir dados de APIs externas com `fetch()`. Agora é o momento de inverter o papel: em vez de consumir uma API, você vai criar a sua própria.

Este módulo apresenta o FastAPI, um framework Python moderno para desenvolvimento de APIs web. Ao final das 8 aulas, você terá uma API funcional com rotas documentadas, validação automática de dados e integração com o front-end construído no Módulo 1.

O que você vai aprender neste módulo

- Capítulo 1 -- HTTP: o protocolo por baixo de tudo
- Capítulo 2 -- Ambiente Python: venv, pip e organização de projetos
- Capítulo 3 -- Introdução ao FastAPI: rotas, parâmetros e Swagger
- Capítulo 4 -- Pydantic: validação e modelagem de dados
- Capítulo 5 -- CRUD completo: POST, PUT, PATCH e DELETE
- Capítulo 6 -- Organização do projeto: routers e estrutura profissional
- Capítulo 7 -- Tratamento de erros e boas práticas
- Capítulo 8 -- Integração com o front-end: CORS e primeiros testes

Por que FastAPI e não Flask ou Django?

Flask: simples e popular, mas exige instalar validação, serialização e documentação separadamente.

Django: completo e poderoso, mas tem uma curva de aprendizado maior e é mais adequado para projetos que usam o ORM e o admin do próprio Django.

FastAPI: moderno, usa tipagem Python nativa, gera documentação Swagger automática, valida dados com Pydantic sem configuração extra e tem performance superior ao Flask.

Para um curso que conecta Python com front-end e banco de dados, o FastAPI é a escolha ideal.

Pré-requisitos deste módulo

Python instalado e funcionando no terminal: `python --version` deve retornar 3.10 ou superior.

VS Code instalado com a extensão Python (Microsoft).

Conhecimento básico de Python: variáveis, funções, listas e dicionários (UC1 e UC2).

GitHub Desktop configurado e repositório dev-sistemas criado.

Capítulo 1

HTTP: O Protocolo por Baixo de Tudo

Toda comunicação na web segue o protocolo HTTP (HyperText Transfer Protocol). Quando você acessa um site, o seu navegador envia uma requisição HTTP para um servidor e o servidor devolve uma resposta HTTP. Entender esse protocolo é essencial para construir APIs que se comportam corretamente.

1.1 O ciclo requisição-resposta

O HTTP segue um modelo cliente-servidor. O cliente faz uma requisição e o servidor processa e responde. É sempre o cliente que inicia a comunicação.

Fluxo completo de uma requisição

1. Cliente (navegador ou JavaScript) monta uma requisição HTTP
2. A requisição contém: método (GET, POST...), URL, headers e opcionalmente um body
3. A requisição viaja pela internet até o servidor
4. O servidor recebe, processa e monta uma resposta HTTP
5. A resposta contém: status code, headers e opcionalmente um body com os dados
6. O cliente recebe e processa a resposta
7. No front-end: o JavaScript lê o JSON e atualiza o DOM

1.2 Métodos HTTP

O método HTTP define a intenção da requisição. Em uma API REST, cada método tem uma semântica específica que deve ser seguida para que o sistema se comporte de forma previsível.

Método	Semântica	Tem body?	Idempotente?	Uso típico
GET	Buscar dados	Não	Sim	Listar recursos, buscar por ID
POST	Criar novo recurso	Sim	Não	Cadastrar usuário, enviar formulário
PUT	Substituir recurso inteiro	Sim	Sim	Atualizar todos os campos de um recurso
PATCH	Atualizar parcialmente	Sim	Sim	Atualizar um ou alguns campos
DELETE	Remover recurso	Não	Sim	Excluir por ID
HEAD	Igual ao GET sem body	Não	Sim	Verificar se recurso existe
OPTIONS	Verificar capacidades	Não	Sim	Usado pelo browser no CORS preflight

Conceito-chave

Idempotente significa: chamar a mesma requisição uma ou várias vezes produz o mesmo resultado no servidor. GET /usuarios/1 sempre retorna o mesmo usuário. DELETE /usuarios/1 remove o usuário na primeira chamada; chamadas seguintes retornam 404, mas o estado final do servidor é o mesmo.

1.3 URLs e endpoints

Uma URL (Uniform Resource Locator) identifica um recurso na web. Em APIs REST, a URL representa um recurso e o método HTTP define a ação sobre ele.

Anatomia de uma URL de API:

```
http://localhost:8000/usuarios/42?ativo=true
^^^^^  ^^^^^^^^^  ^^^  ^^^^^^^^^  ^^^^^^^^^^
scheme  host        port  path       query string

scheme:      http ou https
host:        localhost (desenvolvimento) ou domínio (produção)
port:        8000 (padrão do Uvicorn); 80=HTTP, 443=HTTPS
path:        /usuarios/42 -- identifica o recurso
query string: ?ativo=true -- parâmetros opcionais de filtro
```

Convenções de nomenclatura de endpoints REST:

- Use substantivos no plural para coleções: /usuarios, /produtos, /pedidos.
- Use o ID na URL para recursos individuais: /usuarios/42.
- Use recursos aninhados para relacionamentos: /usuarios/42/pedidos.
- Não use verbos na URL: GET /usuarios é correto. GET /buscarUsuarios é errado.
- Use letras minúsculas e hifens: /produtos-em-estoque, não /ProdutosEmEstoque.

1.4 Status codes HTTP

O status code é um número de três dígitos que indica o resultado da requisição. Usar o status code correto é fundamental: uma API que retorna 200 em caso de erro engana o cliente.

2xx -- Sucesso

Código	Nome	Quando usar
200	OK	Requisição bem-sucedida. Usado para GET, PUT e PATCH.
201	Created	Recurso criado com sucesso. Usado após POST bem-sucedido.
204	No Content	Sucesso sem corpo de resposta. Usado após DELETE.
206	Partial Content	Retorno parcial. Usado em paginação ou download por partes.

4xx -- Erros do cliente

Código	Nome	Quando usar
400	Bad Request	Dados inválidos: tipo errado, campo faltando, regra de negócio violada.
401	Unauthorized	Não autenticado: token ausente, expirado ou inválido.
403	Forbidden	Autenticado, mas sem permissão para o recurso.
404	Not Found	Recurso não encontrado: /usuarios/999 quando o ID não existe.
405	Method Not Allowed	Método HTTP não permitido nessa rota.
409	Conflict	Conflito: tentar criar usuário com e-mail já cadastrado.
422	Unprocessable Entity	Validação falhou. FastAPI retorna este automaticamente.
429	Too Many Requests	Limite de requisições excedido (rate limiting).

5xx -- Erros do servidor

Código	Nome	Quando usar
500	Internal Server Error	Exceção não tratada no servidor. Nunca deve chegar ao cliente.
502	Bad Gateway	Servidor intermediário recebeu resposta inválida.
503	Service Unavailable	Servidor temporariamente indisponível (manutenção, sobrecarga).
504	Gateway Timeout	Servidor demorou demais para responder.

1.5 Headers HTTP

Headers (cabeçalhos) são metadados que acompanham a requisição e a resposta. Informam o formato dos dados, o tipo de autenticação, o idioma e outras configurações.

Header	Tipo	Descrição
Content-Type	Req/Resp	Formato do corpo. application/json para APIs REST.
Accept	Req	Formato aceito pelo cliente. Geralmente application/json.
Authorization	Req	Token de autenticação. Formato: Bearer eyJhbGci...
Cache-Control	Resp	Controla o cache do cliente. no-cache desativa o cache.
Access-Control-Allow-Origin	Resp	CORS: quais origens podem acessar a API.
X-Total-Count	Resp	Convenção para informar o total de registros em paginação.

1.6 Exercícios de fixação

1. Qual o status code correto para cada situação: listar todos os usuários com sucesso, criar um usuário, excluir um usuário, buscar um ID que não existe, enviar um campo obrigatório em branco?
2. Explique a diferença entre PUT e PATCH com um exemplo prático de atualização de um perfil de usuário.
3. Uma API retorna 200 com o body {'erro': 'usuário não encontrado'}. Qual o problema com essa abordagem?
4. Por que não devemos usar verbos na URL de uma API REST? Reescreva corretamente: POST /criarUsuario, GET /buscarProdutos, DELETE /removerPedido/5.
5. O que é idempotência? Por que o DELETE é considerado idempotente mesmo que a segunda chamada retorne 404?